M2PGi – M2M

Pr. Olivier Gruber

(olivier.gruber@univ-grenoble-alpes.fr)

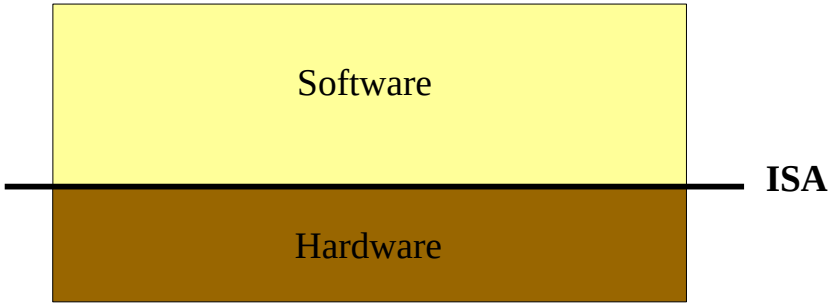
Laboratoire d'Informatique de Grenoble

Université de Grenoble-Alpes

© Pr. Olivier Gruber

Today

- Hardware Support for Interrupts
- Top/Bottom Programming Model



© Pr. Olivier Gruber

Button-Led Example – Polling with or without events...

```
uint8_t button_get_status(void* bar);
void led_on(void* bar);
void led_off(void* bar);
```

Messy one-loop code...

```
void start() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  for (;;) {
    uint8_t status;
    status = button_get_status(BUTTON_BAR);
    if (status & 0x01)
      led_on(LED_BAR);
    else
      led_off(LED_BAR);
  }
}
```

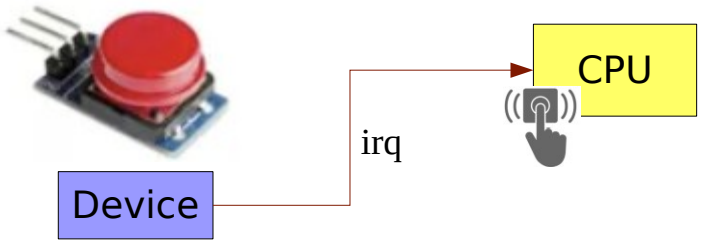
Cleaner event-oriented code...

```
static
void bl_periodic(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
  event_post(bl_periodic,NULL,100);
}


void bl_init() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
  event_post(bl_periodic,NULL,0);
}
```

But still polling...

© Pr. Olivier Gruber

Interrupts – What we wish for...



IRQ: Interrupt ReQuest

Hardware

Software

```
void _start() {
  bl_init();

  for (;;) {
    halt();
  }
}
```

Function to be called when the button is pressed, and only then.

```
static
void bl_handler(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}


void bl_init() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);
}
```

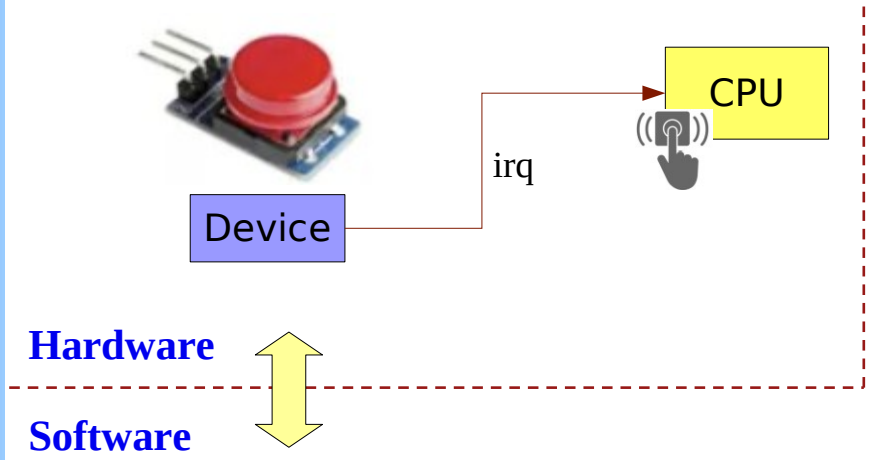
But how will that work exactly?

Let's try to understand it with this simple sleeping loop.

Considering events is too hard of first step...

© Pr. Olivier Gruber

Interrupts – Using Handlers




```
uint8_t button_get_status(void* bar);

void button_enable_interrupt(void (*handler)(void*),
                             void* cookie);

void button_disable_interrupt();
```

```
void core_enable_interrupts();
void core_disable_interrupts();

void _start() {
  bl_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

enable the core to be interrupted
before forcing it to sleep...

Handler
for the button
interrupt

```
static
void bl_handler(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}

void bl_init() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);

  button_enable_interrupt(bl_handler,
                          null);
}
```

enable the button interrupt
and associate the given handler

© Pr. Olivier Gruber

Handlers Execute with Disabled Interrupts


```
void core_enable_interrupts();
void core_disable_interrupts();

void _start() {
  bl_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
uint8_t button_get_status(void* bar);

void button_enable_interrupt(void (*handler)(void*),
                             void* cookie);
void button_disable_interrupt();
```

Upon the interrupt from the button,
the core wakes up,
executes the proper handler,
than returns from "halt()",
loops again and goes back to sleep

```
static
void bl_handler(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}

void bl_init() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);

  button_enable_interrupt(bl_handler,
                          null);
}
```

IMPORTANT: interrupts are disabled
while executing a handler,
automatically re-enabled
after at the core.

© Pr. Olivier Gruber

Another Example – Echoing the UART0


```
void core_enable_interrupts();
void core_disable_interrupts();


void _start() {
  echo_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
uint8_t uart_receive(void* uart);
void    uart_send(void* uart, uint8_t byte);


void uart_enable_interrupt(void (*handler)(void*),
                           void* cookie);

void uart_disable_interrupt();
```

What do we think?

Remember: interrupts are disabled while executing a handler

```
static
void echo_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  if (code != 0)
    uart_send(UART0, code);
}

void echo_init() {
  uart_init(UART0);
  uart_enable_interrupt(echo_handler,
                        null);
}
```

© Pr. Olivier Gruber

Another Example – Echoing the UART0


```
void core_enable_interrupts();
void core_disable_interrupts();


void _start() {
  echo_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
uint8_t uart_receive(void* uart);
void    uart_send(void* uart, uint8_t byte);


void uart_enable_interrupt(void (*handler)(void*),
                           void* cookie);

void uart_disable_interrupt();
```

Not great…

The send operation may block…
so we should **never** call such operations
from a handler…

But let's try first to understand
how to get this code working
in terms of

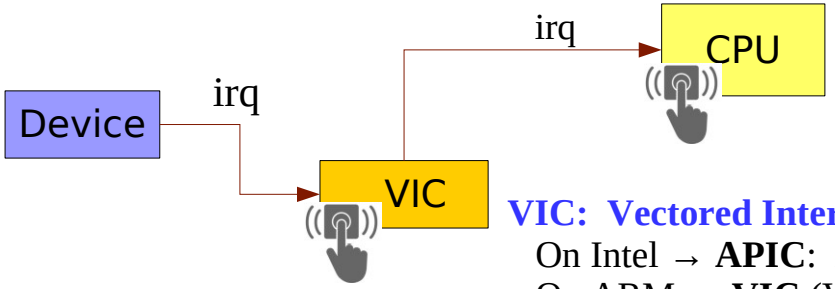
- sleeping loop
- enabling hardware interrupts
- invoked handler on interrupt

```
static
void echo_handler(void* cookie) {
  uint8_t code = uart_receive(UART0);
  if (code != 0)
    uart_send(UART0, code);
}

void echo_init() {
  uart_init(UART0);
  uart_enable_interrupt(echo_handler,
                        null);
}
```

© Pr. Olivier Gruber

Interrupts – Hardware Overview

- When should the processor react to interrupts?
- What should happen?
- How do we manage many devices?



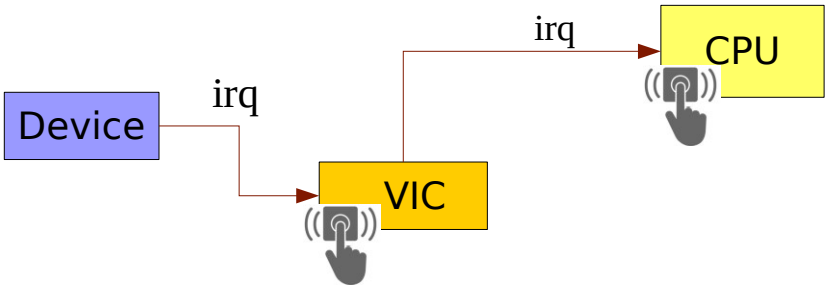
VIC: Vectored Interrupt Controller
On Intel → **APIC**: Advanced Programmable Interrupt Controller
On ARM → **VIC** (Vectored Interrupt Controller)
or **GIC (**Generic Interrupt Controller)

Enable interrupts at devices…
Enable interrupts at the VIC…
Enable interrupts at the CPU...

© Pr. Olivier Gruber

Processor – Interrupt Support

- Processor support
  - New instruction: halt[(1)(2)]
  - New processor pin: interrupt signal, enabled or disabled
  - Interrupts through the exception vector in memory at 0x0000-0000
  - Interrupts can be disabled or enabled



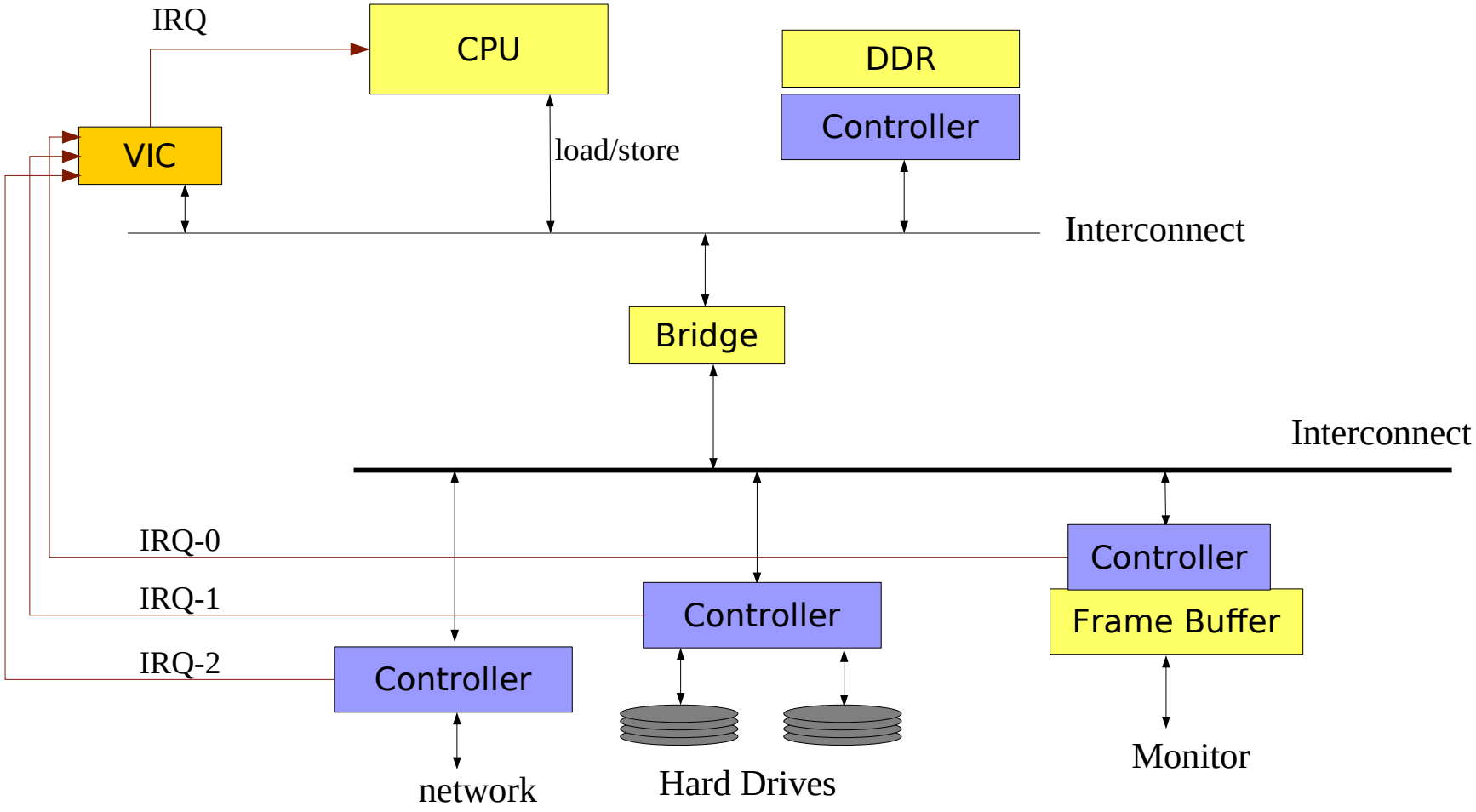

```
void core_enable_interrupts();
void core_disable_interrupts();

void _start() {
  ...
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

(1): Halt instruction has different names for different processors
For example: WFI or WFE for ARM processors

(2): Halts the processor only if there are no pending interrupts and wakes up as soon as an interrupt is pending.

© Pr. Olivier Gruber

Hardware Interrupt Support



PIC: Programmable Interrupt Controller

A chipset to memorize and multiplex interrupt requests from hardware devices.

Configured and controlled through mmio registers, like any other controller

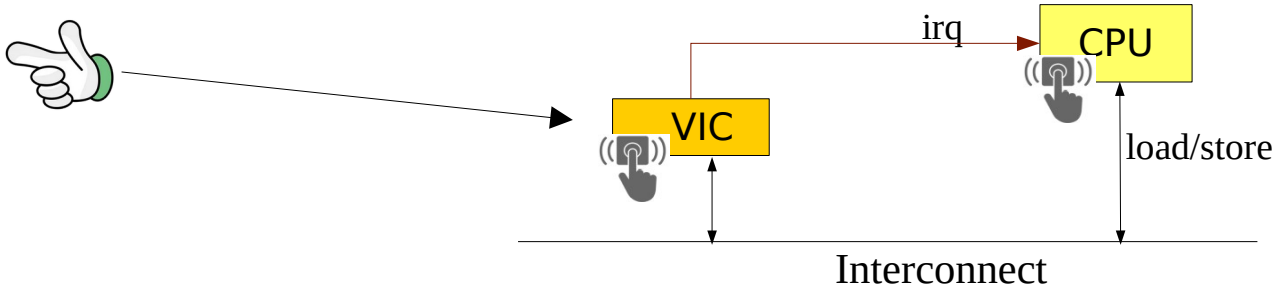
© Pr. Olivier Gruber

Enabling Interrupts

Enabling at the VIC…

Enabling at the CPU

Enabling at the device



PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual

- plugged on the interconnect like other devices/peripherals
- has mmio hardware registers

Chapter 3 **Programmer's Model**

3.1 About the programmer's model 3-2
3.2 Summary of VIC registers 3-3
3.3 Register descriptions 3-6
3.4 Interrupt latency 3-18
3.5 Interrupt priority 3-21

© Pr. Olivier Gruber

PL190 – Registers

PrimeCell Vectored Interrupt Controller (PL190) Technical Reference Manual

Chapter 3 **Programmer's Model**

3.1 About the programmer's model 3-2
3.2 Summary of VIC registers 3-3
3.3 Register descriptions 3-6
3.4 Interrupt latency 3-18
3.5 Interrupt priority 3-21

Table 3-1 VIC register summary

Register	Address offset	Type	Reset value	Description
VICIRQSTATUS	0x000	RO	0x00000000	See *IRQ Status Register* on page 3-6
VICFIQSTATUS	0x004	RO	0x00000000	See *FIQ Status Register* on page 3-6
VICRAWINTR	0x008	RO	-	See *Raw Interrupt Status Register* on page 3-6
VICINTSELECT	0x00C	R/W	0x00000000	See *Interrupt Select Register* on page 3-7
VICINTENABLE	0x010	R/W	0x00000000	See *Interrupt Enable Register* on page 3-7
VICINTENCLEAR	0x014	Write	-	See *Interrupt Enable Clear Register* on page 3-7

© Pr. Olivier Gruber

PL190 – Registers – Enabling / Masking Interrupts

3.3.5 Interrupt Enable Register

The read/write VICINTENABLE Register, with address offset of `0x010`, enables the interrupt request lines, by masking the interrupt sources for the IRQ interrupt. Table 3-6 lists the bit assignments for this register.

Table 3-6 VICINTENABLE Register bit assignments

Bits	Name	Function
[31:0]	IntEnable	Enables the interrupt request lines: 1 = Interrupt enabled. Enables interrupt request to processor. 0 = Interrupt disabled.

"enable by masking"… typical hardware lingo

© Pr. Olivier Gruber

PL190 – Registers – Enabling / Masking Interrupts

3.3.5 Interrupt Enable Register

The read/write VICINTENABLE Register, with address offset of `0x010`, enables the interrupt request lines, by masking the interrupt sources for the IRQ interrupt. Table 3-6 lists the bit assignments for this register.

Table 3-6 VICINTENABLE Register bit assignments

Bits	Name	Function
[31:0]	IntEnable	Enables the interrupt request lines: 1 = Interrupt enabled. Enables interrupt request to processor. 0 = Interrupt disabled. On reset, all interrupts are disabled. A HIGH bit sets the corresponding bit in the VICINTENABLE Register. A LOW bit has no effect.

"enable by masking"… typical hardware lingo

So writing 1s enables…
And writing 0s has no effect…

So how do we clear the interrupt mask?

© Pr. Olivier Gruber

PL190 – Registers – Enable or Mask – Clear or Unmask

3.3.5 Interrupt Enable Register

The read/write VICINTENABLE Register, with address offset of `0x010`, enables the interrupt request lines, by masking the interrupt sources for the IRQ interrupt. Table 3-6 lists the bit assignments for this register.

Table 3-6 VICINTENABLE Register bit assignments

Bits	Name	Function
[31:0]	IntEnable	Enables the interrupt request lines: 1 = Interrupt enabled. Enables interrupt request to processor. 0 = Interrupt disabled. On reset, all interrupts are disabled. A HIGH bit sets the corresponding bit in the VICINTENABLE Register. A LOW bit has no effect.

3.3.6 Interrupt Enable Clear Register

The write-only VICINTENCLEAR Register, with address offset of `0x014`, clears bits in the VICIntEnable Register. Table 3-7 lists the bit assignments for this register.

Table 3-7 VICINTENCLEAR Register bit assignments

Bits	Name	Function
[31:0]	IntEnable Clear	Clears bits in the VICINTENABLE Register. A HIGH bit clears the corresponding bit in the VICINTENABLE Register. A LOW bit has no effect.

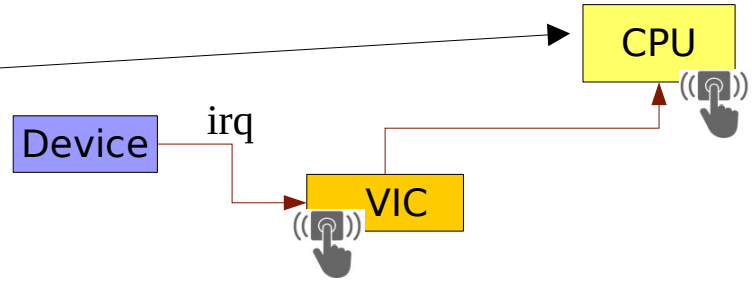
© Pr. Olivier Gruber

Enabling Interrupts

Enabling at the VIC…

Enabling at the CPU

Enabling at the device…



Processors, ***rather cores***, have special registers/instructions to enable/disable interrupts at the core level

```
/* Update the Current Program Status Register (cpsr),
 * enabling IRQs on the processor by clearing the bit 0x80
 */
 mrs r0, cpsr
 bic r0, r0, #0x80
 msr cpsr, r0
```

Instruction: bic (BitClear)
bic r0, r0, 0x80

→ r0 = r0 & ~0x80

Effectively clearing the bit

→ enables the interrupts

```
/* Update the Current Program Status Register (cpsr),
 * disabling IRQs on the processor by setting the bit 0x80
 */
 mrs r0, cpsr
 orr r0, r0, #0x80
 msr cpsr, r0
```

Instruction: bic (BitClear)
orr r0, r0, 0x80

→ r0 = r0 | 0x80

Effectively setting the bit

→ disable the interrupts

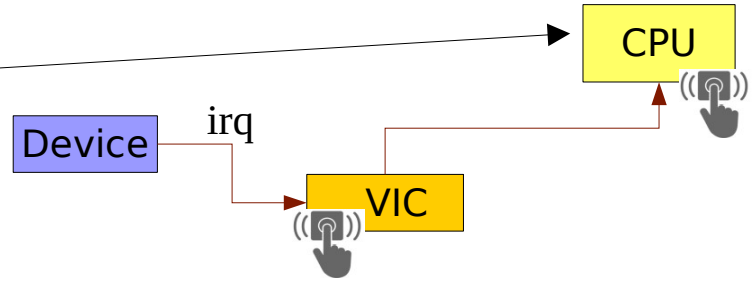
© Pr. Olivier Gruber

Enabling Interrupts

Enabling at the VIC…

Enabling at the CPU

Enabling at the device…



Processors, ***rather cores***, have special registers/instructions to enable/disable interrupts at the core level

```
/* Update the Current Program Status Register (cpsr),
 * enabling IRQs on the processor by clearing the bit 0x80
 */
 mrs r0, cpsr
 bic r0, r0, #0x80
 msr cpsr, r0
```

```
/* Update the Current Program Status Register (cpsr),
 * disabling IRQs on the processor by setting the bit 0x80
 */
 mrs r0, cpsr
 orr r0, r0, #0x80
 msr cpsr, r0
```

Clearing the bit

→ enables the interrupts

Setting the bit

→ disable the interrupts

This is the opposite than with the VIC…

Because we are not speaking of a ***mask*** here…

Welcome to the hardware world!!!

© Pr. Olivier Gruber

ARM Processor Modes – Cortex-A8

Privileged modes

Exception modes

User	System	Supervisor	Abort	Undefined	Interrupt	Fast interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

Banked registers
lr: link-return register (r14)
sp: stack-pointer register (r13)

Current Program Status Register

Saved Program Status Register

indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode

© Pr. Olivier Gruber

ARM Processor Stacks – Cortex-A8

- ARM Processor like the Cortex-A8
  - Do have different modes with banked registers
    - Banked stack-pointer register (**sp** = r13)
    - Banked link-return register (**lr** = r14)
  - This means a different stack per mode
    - We will use two modes `System` and `Interrupt`
    - So we will need two stacks
  - Disable interrupts
    - Disable interrupts at the core until our software is ready for them...

```
/*
* Set the core in the SYS_MODE, with both IRQs and FIQs
* disabled, also set the stack for the SYS_MODE
*/
msr    cpsr_c, #(CPSR_SYS_MODE |
                 CPSR_IRQ_FLAG | CPSR_FIQ_FLAG)
/* set the C stack pointer */
ldr    sp,=stack_top
```

```
/* Standard definitions of Mode bits
   in the Current Program Status Register */

   .equ    CPSR_USR_MODE,       0x10
   .equ    CPSR_FIQ_MODE,       0x11
   .equ    CPSR_IRQ_MODE,       0x12
   .equ    CPSR_SVC_MODE,       0x13
   .equ    CPSR_ABT_MODE,       0x17
   .equ    CPSR_UND_MODE,       0x1B
   .equ    CPSR_SYS_MODE,       0x1F

   /* when set, IRQs are disabled. */
   .equ    CPSR_IRQ_FLAG,         0x80
   /* when set, FIQs are disabled. */
   .equ    CPSR_FIQ_FLAG,         0x40
```

© Pr. Olivier Gruber

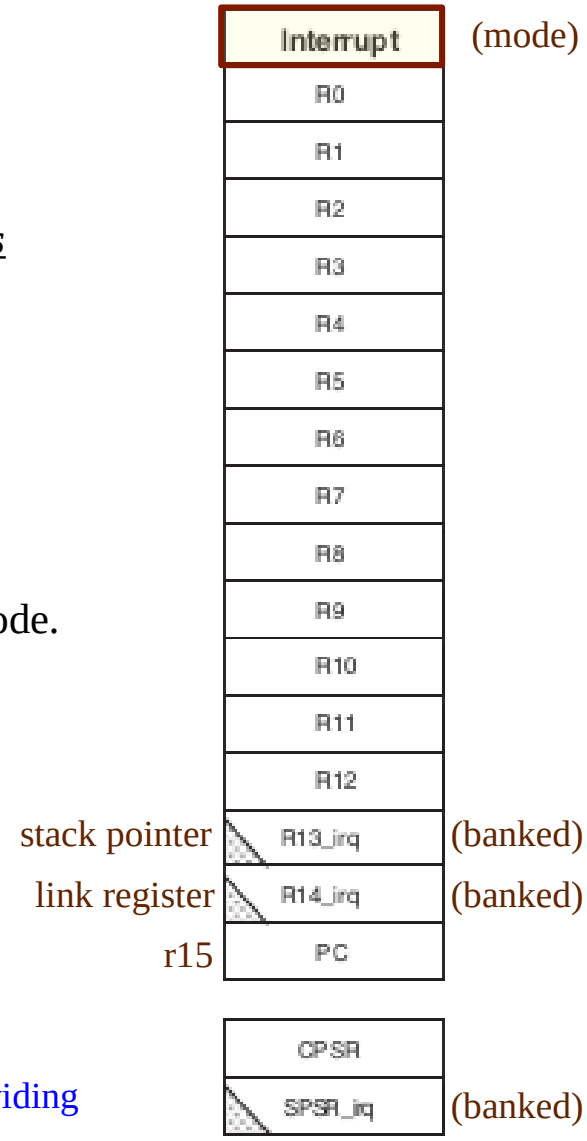
Interrupt Processing – Processor perspective

Interrupt steps taken by the processor:

- Save the CPSR into the banked ***SPSR_irq***
- Save **R15** into the banked ***R14_irq***
- Switch the processor in ***interrupt mode, with <u>disabled interrupts</u>***
- Set **R15** to **<u>0x18</u>** and execute the instruction there

But what is the value of the banked register R13_irq (sp)?

- It must be set as part of the system initialization
- Done after a hardware reset
- There is a ***Current Program Status Register*** that controls the mode.



Note: CPSR: Current Program Status Register.
SPSR: Saved Program Status Register.

One of the most important system registers, controlling your processor and providing status bits. In particular, the current operation mode of your processor.

© Pr. Olivier Gruber

Interrupt Processing – Hardware / Software Boundary

- Exception vector in memory
  - Interrupts are “one kind of exceptions”
  - Failures are others

This is reset

These are traps, as a side effect of executing an instruction.

This is for hardware interrupts from devices

```
; hardware reset address = 0x00000000

0x00    br    _reset
0x04    br    _undef_inst
0x08    br    _soft_irq
0x0c    br    _prefetch_abort
0x10    br    _data_abort
0x14    br    _not_used
0x18    br    _isr
0x1c    br    _fisr  // we will not use these
```

traps

This is the software reaction/continuation to the hardware having reset...

```
_reset:
   ...
```

The *Interrupt Service Routine* processes all interrupts...

```
_isr:
   ...
```

© Pr. Olivier Gruber

Interrupt Processing – Hardware / Software Boundary

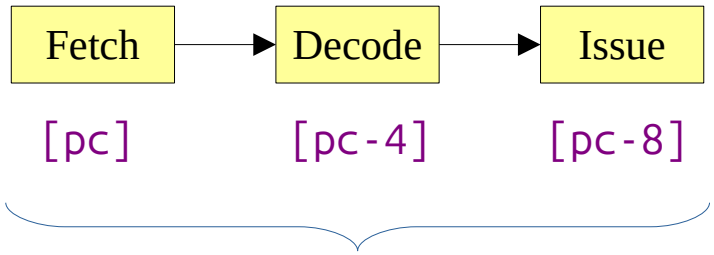
More flexible coding of the same thing:

- Use a special load instruction, relative to the address in the pc register
  `ldr pc, [ pc, #offset ]`

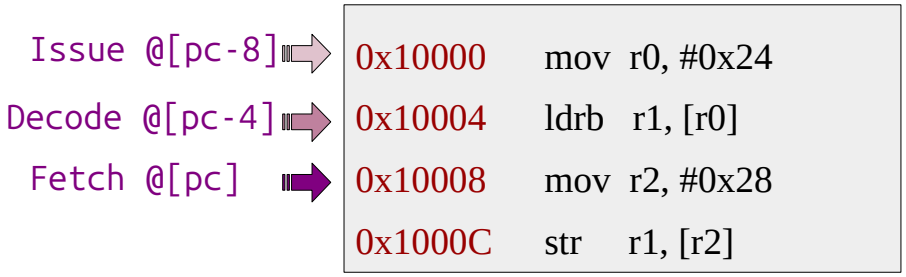
- Removes the constraint of having the called function near by, because a branch instruction can have only a small offset encoded in the instruction.

Nota Bene:
Because of the ***3-stage pipeline***, the pc is 8 bytes ahead on Cortex-A8,
hence: `ldr pc, [pc,#0x18]`
and not: `ldr pc, [pc,#0x20]`

```
; hardware reset address = 0x00000000

0x00    ldr  pc, [pc,#0x18]
0x04    ldr  pc, [pc,#0x18]
0x08    ldr  pc, [pc,#0x18]
0x0c    ldr  pc, [pc,#0x18]
0x10    ldr  pc, [pc,#0x18]
0x14    ldr  pc, [pc,#0x18]
0x18    ldr  pc, [pc,#0x18]
0x1c    ldr  pc, [pc,#0x18]

0x20    .word _reset
0x24    .word _undefined_instruction
0x28    .word _software_interrupt
0x2c    .word _prefetch_abort
0x30    .word _data_abort
0x34    .word not_used
0x38    .word _isr
0x3c    .word _fisr
```



Parallel Pipeline Stages



© Pr. Olivier Gruber

Interrupt Service Routine

- Interrupt Service Routine (ISR)
  - The simplest interrupt service routine, up-calling C code
  - Then returning to normal execution

```
_isr: ; interrupt service routine
    sub lr,lr,#4                    ; adjust return address (see Cortex-A8 documentation)
    stmfd   sp!, {r0-r12, lr}       ; save registers with link register last
    bl isr                          ; now call the C function interrupt_service_routine
    ldmfd    sp!, {r0-r12, pc}^     ; back from C…
                                    ; restore all registers, including pc from saved lr
```

Note: use store/load multiple instructions (stm/ldm) for stack manipulations,
stacks can be descending (stmfd/ldmfd) or ascending (stmfa/ldmfa), stack pointer (sp) is r13.
The presence of '!' specifies that the final address is written back into r13.

Note: ***The ^ qualifier on ldmfd specifies that the CPSR is restored from the SPSR***.
CPSR: Current Program Status Register.
SPSR: Saved Program Status Register.

© Pr. Olivier Gruber

Interrupt Service Routine

- Cortex-a8 (ARM DDI 0344E), page 2-35

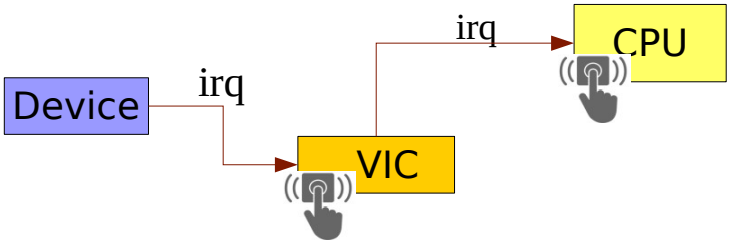
Table 2-12 Exception entry and exit

Exception or entry	Return instruction	Previous state		Notes
		ARM r14_x	Thumb r14_x	
SVC	MOVS PC, R14_svc	PC + 4	PC+2	Where the PC is the address of the SVC, SMC, or Undefined instruction
SMC	MOVS PC, R14_mon	PC + 4	-	
UNDEF	MOVS PC, R14_und	PC + 4	PC+2	
PABT	SUBS PC, R14_abt, #4	PC + 4	PC+4	Where the PC is the address of instruction that had the prefetch abort
FIQ	SUBS PC, R14_fiq, #4	PC + 4	PC+4	Where the PC is the address of the instruction that was not executed because the FIQ or IRQ took priority
IRQ	SUBS PC, R14_irq, #4	PC + 4	PC+4	
DABT	SUBS PC, R14_abt, #8	PC + 8	PC+8	Where the PC is the address of the load or store instruction that generated the data abort
RESET	-	-	-	The value saved in r14_svc on reset is Unpredictable
BKPT	SUBS PC, R14_abt, #4	PC + 4	PC+4	Software breakpoint

Copyright © 2006-2008 ARM Limited. All rights reserved.

© Pr. Olivier Gruber

Summary



The instruction "halt" such as ***WFI*** or ***WFE*** for ARM processors

Vectored Interrupt Controller: (architected state per interrupt)

Trapping for the Interrupt Processing by the processor

FIQ	SUBS PC, R14_fiq, #4	PC + 4	PC+4	Where the PC is the address of the instruction that was not executed because the FIQ or IRQ took priority
IRQ	SUBS PC, R14_irq, #4	PC + 4	PC+4	

	Privileged modes					
		Exception modes				
User	System	Supervisor	Abort	Undefined	Interrupt	Fast interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode

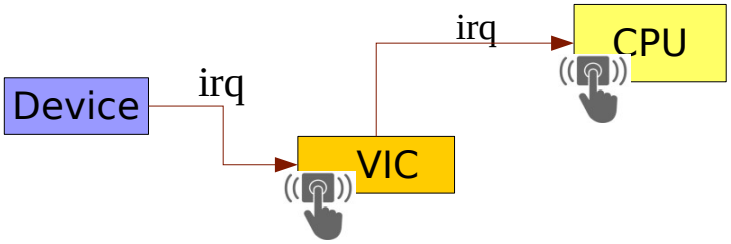
```
0x00    br    _reset
0x04    br    _undef_inst
0x08    br    _soft_irq
0x0c    br    _prefetch_abort
0x10    br    _data_abort
0x14    br    _not_used
0x18    br    _isr
0x1c    br    _fisr
```

```
_isr:
    sub     lr,lr,#4
    stmfd   sp!, {r0-r12, lr}
    bl isr  // C function call
    ldmfd   sp!, {r0-r12, pc}^
```

The C function "isr()" is the real processing of the interrupt requests, attending the requesting devices

© Pr. Olivier Gruber

Summary



The instruction "halt" such as ***WFI*** or ***WFE*** for ARM processors

Vectored Interrupt Controller: (architected state per interrupt)

Trapping for the Interrupt Processing by the processor

FIQ	SUBS PC, R14_fiq, #4	PC + 4	PC+4	Where the PC is the address of the instruction that was not executed because the FIQ or IRQ took priority
IRQ	SUBS PC, R14_irq, #4	PC + 4	PC+4	

	Privileged modes					
		Exception modes				
User	System	Supervisor	Abort	Undefined	Interrupt	Fast interrupt
R0	R0	R0	R0	R0	R0	R0
R1	R1	R1	R1	R1	R1	R1
R2	R2	R2	R2	R2	R2	R2
R3	R3	R3	R3	R3	R3	R3
R4	R4	R4	R4	R4	R4	R4
R5	R5	R5	R5	R5	R5	R5
R6	R6	R6	R6	R6	R6	R6
R7	R7	R7	R7	R7	R7	R7
R8	R8	R8	R8	R8	R8	R8_fiq
R9	R9	R9	R9	R9	R9	R9_fiq
R10	R10	R10	R10	R10	R10	R10_fiq
R11	R11	R11	R11	R11	R11	R11_fiq
R12	R12	R12	R12	R12	R12	R12_fiq
R13	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
R14	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
PC	PC	PC	PC	PC	PC	PC
CPSR	CPSR	CPSR	CPSR	CPSR	CPSR	CPSR
		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

indicates that the normal register used by User or System mode has been replaced by an alternative register specific to the exception mode

```
0x00    br    _reset
0x04    br    _undef_inst
0x08    br    _soft_irq
0x0c    br    _prefetch_abort
0x10    br    _data_abort
0x14    br    _not_used
0x18    br    _isr
0x1c    br    _fisr
```

But all this works <u>only</u> if interrupts are enabled

- ***at the VIC***
- ***at the CPU***
- ***at devices…***

How?

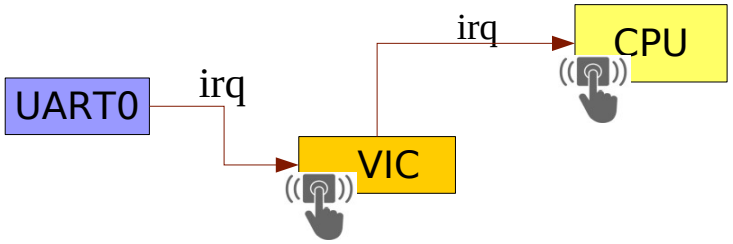
© Pr. Olivier Gruber

Enabling Interrupts

Enabling at the VIC

Enabling at the CPU

Enabling at the device…



Example: *the UART…*

- where is it mapped?

- which chipset?

- what are its mmio registers?

© Pr. Olivier Gruber

Where Is the UART Mapped?

Versatile AB/PB User Guide – 4.1 Memory Map

Table 4-1 Memory map

Peripheral	Location	Interrupt[a] PIC and SIC	Address	Region size
MPMC Chip Select 0. Normally the bottom 64MB of SDRAM (During boot remapping, this can be NOR flash, Disk-on-Chip, or static expansion memory)	Board	PIC21, SIC21	0x00000000–0x03FFFFFF	64MB

Table 4-1 Memory map (continued)

Peripheral	Location	Interrupt[a] PIC and SIC	Address	Region size
Smart Card 0 Interface	Dev. chip	PIC 15	0x101F0000–0x101F0FFF	4KB
UART 0 Interface	Dev. chip	PIC 12	0x101F1000–0x101F1FFF	4KB
UART 1 Interface	Dev. chip	PIC 13	0x101F2000–0x101F2FFF	4KB
UART 2 Interface	Dev. chip	PIC 14	0x101F3000–0x101F3FFF	4KB

© Pr. Olivier Gruber

UART – Which Chipset?

Versatile AB/PB User Guide – UART Section

PL011 PrimeCell

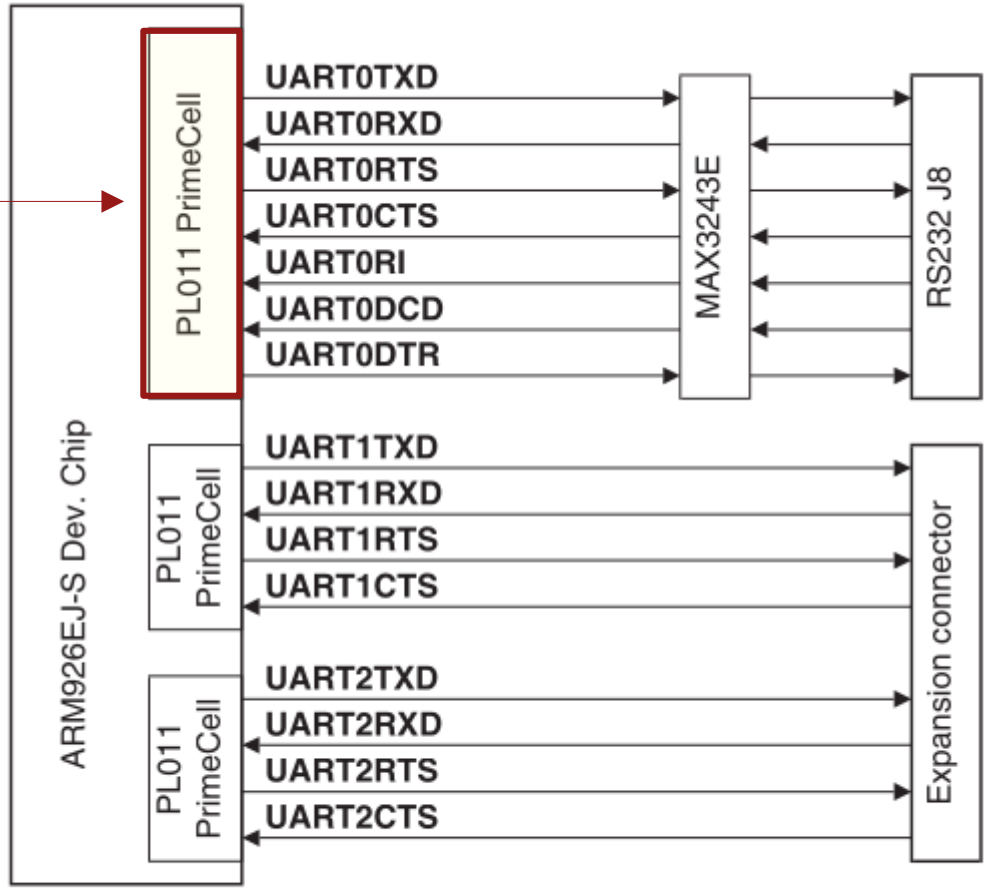
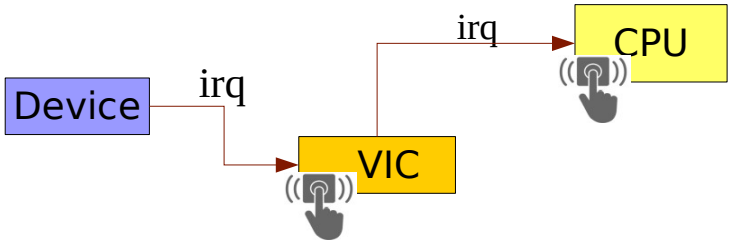


Figure 3-31 UARTs block diagram

© Pr. Olivier Gruber

Enabling Interrupts at the Device



PrimeCell UART (PL011) Technical Reference Manual

Chapter 3 **Programmers Model**

3.1 About the programmers model 3-2

ARM DDI 0183G *Copyright © 2000, 2001, 2005, 2007 ARM Limited. All rights reserved.* v

Contents

3.2 Summary of registers 3-3
3.3 Register descriptions 3-5

© Pr. Olivier Gruber

PL011 Registers – Data & Status

3.2 Summary of registers

Table 3-1 lists the UART registers.

Table 3-1 UART register summary

Offset	Name	Type	Reset	Width	Description
0x000	UARTDR	RW	0x---	12/8	*Data Register, UARTDR* on page 3-5
0x004	UARTRSR/ UARTECR	RW	0x0	4/0	*Receive Status Register/Error Clear Register, UARTRSR/UARTECR* on page 3-6
0x008-0x014	-	-	-	-	Reserved
0x018	UARTFR	RO	0b-10010---	9	*Flag Register, UARTFR* on page 3-8
0x01C	-	-	-	-	Reserved

© Pr. Olivier Gruber

PL011 Registers – Interrupt-related Registers

PrimeCell UART (PL011) Technical Reference Manual

0x038	UARTIMSC	RW	0x000	11	*Interrupt Mask Set/Clear Register, UARTIMSC* on page 3-17
0x03C	UARTRIS	RO	0x00-	11	*Raw Interrupt Status Register, UARTRIS* on page 3-19
0x040	UARTMIS	RO	0x00-	11	*Masked Interrupt Status Register, UARTMIS* on page 3-20
0x044	UARTICR	WO	-	11	*Interrupt Clear Register, UARTICR* on page 3-21

Interrupt Mask Set/Clear Register (UARTIMSC)

Yet another way… the set/clear way!

The UARTIMSC Register is the interrupt mask ***set/clear*** register.

- On a **read** this register returns the current value of the mask for interrupts.
- On a ***write of 1*** to a particular bit, it ***sets*** the corresponding mask of that interrupt.
- On a ***write of 0*** to a particular bit, it ***clears*** the corresponding mask.

All the bits are cleared when reset, so all interrupts are disabled after a reset.

Nota Bene: a masked interrupt is an enabled one.
(the hardware does a binary-and operation with the mask)

© Pr. Olivier Gruber

PL011 Registers – UARTIMSC Bit Field

Table 3-14 UARTIMSC Register

Bits	Name	Function
15:11	-	Reserved, read as zero, do not modify.
10	OEIM	Overrun error interrupt mask. A read returns the current mask for the **UARTOEINTR** interrupt. On a write of 1, the mask of the **UARTOEINTR** interrupt is set. A write of 0 clears the mask.
9	BEIM	Break error interrupt mask. A read returns the current mask for the **UARTBEINTR** interrupt. On a write of 1, the mask of the **UARTBEINTR** interrupt is set. A write of 0 clears the mask.
8	PEIM	Parity error interrupt mask. A read returns the current mask for the **UARTPEINTR** interrupt. On a write of 1, the mask of the **UARTPEINTR** interrupt is set. A write of 0 clears the mask.
7	FEIM	Framing error interrupt mask. A read returns the current mask for the **UARTFEINTR** interrupt. On a write of 1, the mask of the **UARTFEINTR** interrupt is set. A write of 0 clears the mask.
6	RTIM	Receive timeout interrupt mask. A read returns the current mask for the **UARTRTINTR** interrupt. On a write of 1, the mask of the **UARTRTINTR** interrupt is set. A write of 0 clears the mask.
5	TXIM	Transmit interrupt mask. A read returns the current mask for the **UARTTXINTR** interrupt. On a write of 1, the mask of the **UARTTXINTR** interrupt is set. A write of 0 clears the mask.
4	RXIM	Receive interrupt mask. A read returns the current mask for the **UARTRXINTR** interrupt. On a write of 1, the mask of the **UARTRXINTR** interrupt is set. A write of 0 clears the mask.
3	DSRMIM	**nUARTDSR** modem interrupt mask. A read returns the current mask for the **UARTDSRINTR** interrupt. On a write of 1, the mask of the **UARTDSRINTR** interrupt is set. A write of 0 clears the mask.
2	DCDMIM	**nUARTDCD** modem interrupt mask. A read returns the current mask for the **UARTDCDINTR** interrupt. On a write of 1, the mask of the **UARTDCDINTR** interrupt is set. A write of 0 clears the mask.
1	CTSMIM	**nUARTCTS** modem interrupt mask. A read returns the current mask for the **UARTCTSINTR** interrupt. On a write of 1, the mask of the **UARTCTSINTR** interrupt is set. A write of 0 clears the mask.
0	RIMIM	**nUARTRI** modem interrupt mask. A read returns the current mask for the **UARTRIINTR** interrupt. On a write of 1, the mask of the **UARTRIINTR** interrupt is set. A write of 0 clears the mask.

Eleven different interrupts…
Two of interest to us: TX / RX

© Pr. Olivier Gruber

PL011 Registers – Interrupt-related Registers

PrimeCell UART (PL011) Technical Reference Manual

0x038	UARTIMSC	RW	0x000	11	*Interrupt Mask Set/Clear Register, UARTIMSC* on page 3-17
0x03C	UARTRIS	RO	0x00-	11	*Raw Interrupt Status Register, UARTRIS* on page 3-19
0x040	UARTMIS	RO	0x00-	11	*Masked Interrupt Status Register, UARTMIS* on page 3-20
0x044	UARTICR	WO	-	11	*Interrupt Clear Register, UARTICR* on page 3-21

Raw/Masked Interrupt Status Registers (UARTRIS & UARTMIS)

Both registers have the same bit field as the UARTIMSC register.

The UARTRIS Register is the ***raw interrupt status*** register. It is ***read-only register***.

The UARTMIS Register is the ***masked interrupt status*** register. It is read-only register.

The status of the raw interrupts (UARTRIS) are masked by the enabled bits (UARTIMSC), giving the masked status (UARTMIS).

© Pr. Olivier Gruber

PL011 Registers – Interrupt-related Registers

PrimeCell UART (PL011) Technical Reference Manual

0x038	UARTIMSC	RW	0x000	11	*Interrupt Mask Set/Clear Register, UARTIMSC* on page 3-17
0x03C	UARTRIS	RO	0x00-	11	*Raw Interrupt Status Register, UARTRIS* on page 3-19
0x040	UARTMIS	RO	0x00-	11	*Masked Interrupt Status Register, UARTMIS* on page 3-20
0x044	UARTICR	WO	-	11	*Interrupt Clear Register, UARTICR* on page 3-21

Interrupt Clear Register (UARTICR)

this is clearing the interrupt not clearing the mask!

The register has the same bit field as the UARTIMSC register.

The UARTICR register is a ***write-only register***.

- Write of 1 to a particular bit, the corresponding interrupt is cleared.
- Write of 0 to a particular bit has no effect.

Nota Bene: clearing an interrupt tells the chipset
that the corresponding interrupt request has been handled by software.
If not cleared, an interrupt will never be requested again.

© Pr. Olivier Gruber

When is the UART requesting an interrupt?

PrimeCell UART (PL011) Technical Reference Manual

3.2 Summary of registers

Table 3-1 lists the UART registers.

Table 3-1 UART register summary

Offset	Name	Type	Reset	Width	Description
0x034	UARTIFLS	RW	0x12	6	*Interrupt FIFO Level Select Register, UARTIFLS* on page 3-17

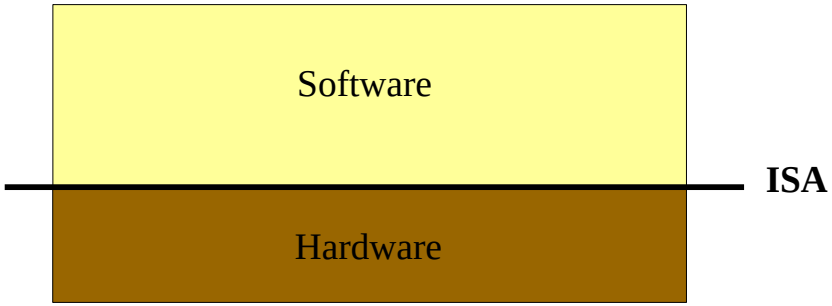
Interrupt FIFO Level Select Register (UARTIFLS)

The interrupts are generated based on ***a transition through a level*** rather than being based on the level. That is, the interrupts are generated when the fill level progresses through the trigger level.

© Pr. Olivier Gruber

Today

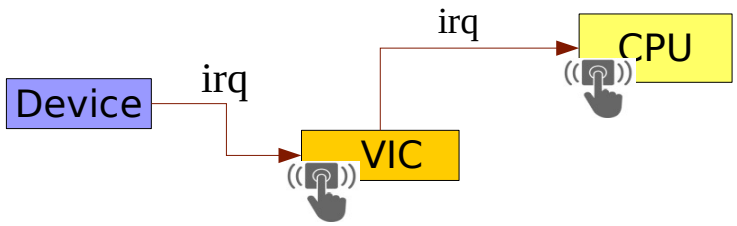
- Analyses of our code
  - Working with peripherals
  - Top-level loop analysis
- Designing solutions
  - Processor Support for Interrupts
  - **Software Support for Interrupts**



© Pr. Olivier Gruber

Interrupt Service Routine – “Who is it?“

- Detect which interrupts are pending
  - Dialog with the VIC/GIC/PIC[1]
  - Using its mmio registers
    - Bit fields for raised interrupts
    - Bit fields for enabled/disabled interrupts
- Call the corresponding interrupt handlers
  - Indexed by the interrupt number
- *Acknowledge the raised interrupts*
  - Tells the device it has been serviced
  - If you do not acknowledged, the device will never raise its interrupt again...

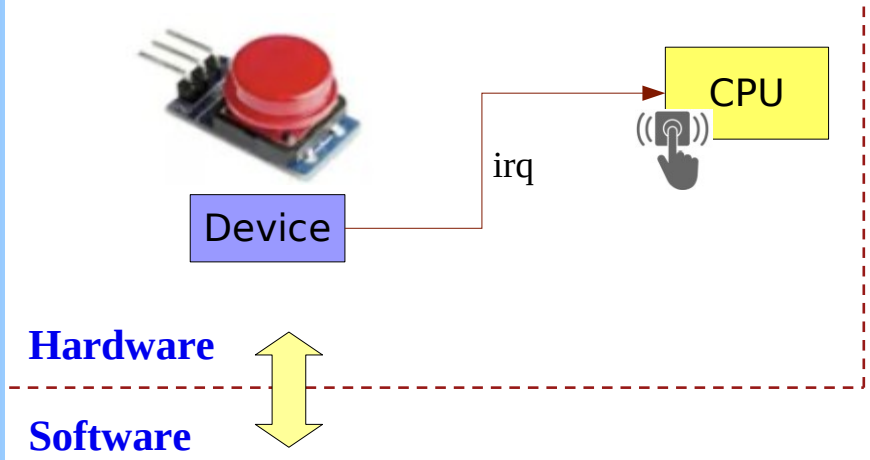



```
.global _isr
_isr:
    ...
    ; now call the interrupt_service_routine
    bl isr
    ...
```

```
struct handler {
  void (*callback)(void*);
  void* cookie;
};

struct handler handlers[MAX_NHANDLERS];

void isr() {
  uint32_t irqs = vic_load_irqs();
  for (uint32_t i=0 ; i<32 ; i++) {
    struct handler_t* handler;
    handler = &handlers[i];
    if (irqs & (1<<i))
      handler->callback(handler->cookie);
  }
  vic_ack_irqs(irqs);
  return;
}
```

(1) GIC: Generic Interrupt Controller (ARM)
APIC: Advanced Programmable Interrupt Controller (Intel)

Going Up – Button Support




```
void vic_enable_interrupt(uint32_t irqno,
                          void (*handler)(void*), void* cookie);
void vic_disable_interrupt(uint32_t irqno);
```

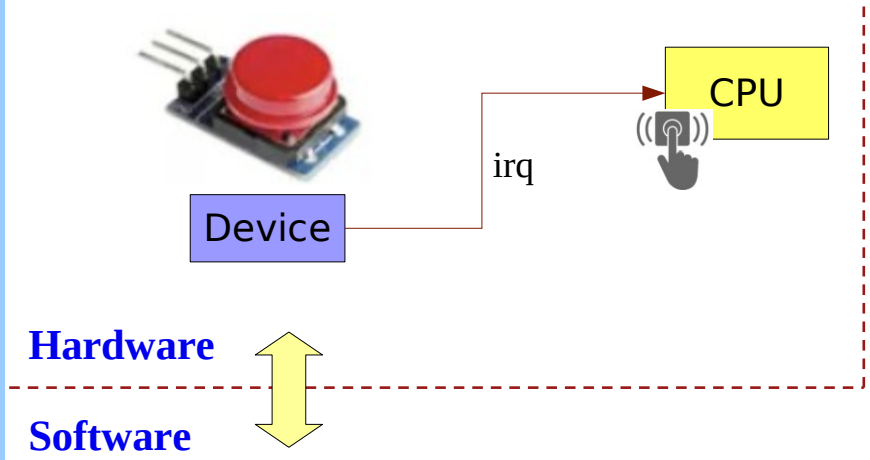
```
void core_enable_interrupts();
void core_disable_interrupts();
```

```
void _start() {
  ...
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
uint8_t button_get_status(void* bar);

void button_enable_interrupt(void (*handler)(void*),
                             void* cookie) {
  vic_enable_interrupt(BUTTON_IRQ,handler,cookie);
  ... // enable interrupt at the button device
}

void button_disable_interrupt() {
  ... // disable interrupt at the button device
  vic_disable_interrupt(BUTTON_IRQ);
}
```

© Pr. Olivier Gruber

Going Up – Button-and-Led Support




```
uint8_t button_get_status(void* bar);

void button_enable_interrupt(void (*handler)(void*),
                             void* cookie);
void button_disable_interrupt();
```

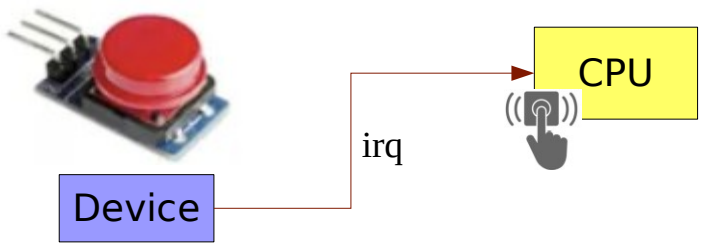
```
void core_enable_interrupts();
void core_disable_interrupts();
```

```
void _start() {
  bl_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
static
void bl_handler(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}

void bl_init() {
  button_init_regs(BUTTON_BAR);
  led_init_regs(LED_BAR);

  button_enable_interrupt(bl_handler,
                          null);
}
```

© Pr. Olivier Gruber

Going Up – Button-and-Led Support



Hardware

Software

```
void _start() {
  bl_init();
  core_enable_interrupts();
  for (;;) {
    halt();
  }
}
```

```
0x00    ldr  pc, [pc,#0x18]
...
0x18    ldr  pc, [pc,#0x18]
0x1c    ldr  pc, [pc,#0x18]
0x20    .word _reset
...
0x38    .word _isr

_isr:
    ...
    bl isr
    ...
```

```
struct handler {
  void (*callback)(void*);
  void* cookie;
};

struct handler handlers[MAX_NHANDLERS];

void isr() {
  uint32_t irqs = vic_load_irqs();
  for (uint32_t i=0 ; i<32 ; i++) {
    struct handler_t* handler;
    handler = &handlers[i];
    if (irqs & (1<<i))
      handler->callback(handler->cookie);
  }
  vic_ack_irqs(irqs);
  return;
}
```

```
static
void bl_handler(void* cookie) {
  uint8_t status;
  status = button_get_status(BUTTON_BAR);
  if (status & 0x01)
    led_on(LED_BAR);
  else
    led_off(LED_BAR);
}
```

© Pr. Olivier Gruber